



IA CORPORATIVA COM JAVA E LANGCHAIN4J

AULA 01 - OVERVIEW E FUNDAMENTOS



OBJETIVOS DA AULA

- ❑ Diferenciar IA Preditiva de IA Generativa.
- ❑ Compreender a fundamentação do curso: o desenvolvedor Java como integrador de IA, não como cientista de dados.
- ❑ Identificar os componentes do stack tecnológico: Quarkus, Langchain4j e Ollama, e o papel de cada um.
- ❑ Visualizar a arquitetura inicial da implementação de referência do curso e seu plano de evolução.



O MUNDO DA IA: DUAS REALIDADES DISTINTAS

O MUNDO DA IA: DUAS REALIDADES DISTINTAS

- ❑ A inteligência artificial no ambiente corporativo se manifesta em duas vertentes principais, com propósitos e capacidades fundamentalmente diferentes.
- ❑ IA Preditiva:
 - ❑ Tecnologia madura, baseada em modelos estatísticos e machine learning clássico.
 - ❑ Objetivo: Analisar dados históricos para prever resultados futuros ou classificar informações.
 - ❑ Saída: Estruturada (um número, uma categoria, uma probabilidade). Ex: score de crédito, detecção de fraude.
- ❑ IA Generativa:
 - ❑ Tecnologia emergente projetada para criar conteúdo novo e original.
 - ❑ Base: Utiliza Large Language Models (LLMs) para gerar textos, imagens, música e código em resposta a um prompt.
 - ❑ Saída: Não estruturada e criativa. Abre possibilidades para automação de comunicação e desenvolvimento de software.

O PAPEL DO DESENVOLVEDOR

O PAPEL DO DESENVOLVEDOR

- ❑ A transição da IA Preditiva para a Generativa eleva o desenvolvedor de um mero consumidor de APIs para um arquiteto de sistemas cognitivos.

Característica	IA Preditiva	IA Generativa
Objetivo Principal	Prever um resultado futuro (classificar, prever)	Criar conteúdo novo e coerente (gerar, resumir)
Tipo de Saída	Estruturada: um número, uma categoria (ex: true/false)	Não estruturada: texto em linguagem natural, código, JSON
Papel do Desenvolvedor	Consumidor de Modelo: Integrar uma API que retorna uma predição.	Arquiteto Cognitivo: Orquestrar o fluxo de informação e o "raciocínio" do LLM.
Exemplo Corporativo	Sistema que prevê a probabilidade de um cliente cancelar a assinatura.	Agente que lê e-mails de clientes e gera uma resposta contextualizada.

O PAPEL DO DESENVOLVEDOR

- ❑ IA Preditiva
 - ❑ O desenvolvedor invocava um endpoint e processava uma resposta estruturada, tratando o modelo como uma “caixa-preta”.
- ❑ IA Generativa
 - ❑ O desafio é mais estratégico. O desenvolvedor precisa projetar todo o fluxo de "pensamento" da aplicação :
 - ❑ Determinar se e quais informações externas são necessárias (ex: RAG).
 - ❑ Formatar essas informações em estruturas e prompts eficazes.
 - ❑ Definir e disponibilizar ferramentas que serão integradas ao LLM.
 - ❑ Validar e utilizar a resposta gerada.

**FOCO EM INTEGRAÇÃO, NÃO
EM CIÊNCIA DE DADOS**



O PAPEL DO DESENVOLVEDOR

- ❑ A grande força do Java está em sua vasta integração dentro do ambiente corporativo
- ❑ O real valor do desenvolvedor Java na era da IA não está em treinar modelos, mas na integração de LLMs pré-treinados com os sistemas corporativos existentes (ERPs, CRMs, etc.).
- ❑ O ecossistema Java é ideal para esta tarefa devido a:
 - ❑ Escalabilidade e Performance: Frameworks como Quarkus são otimizados para nuvem, com baixo consumo de memória e startup rápido.
 - ❑ Segurança: Modelo de segurança maduro para manuseio de dados sensíveis.
 - ❑ Ecossistema Maduro: Vastas bibliotecas para conectividade, transações e resiliência.

A DEMOCRATIZAÇÃO DA IA

A DEMOCRATIZAÇÃO DA IA

- ❑ Ferramentas como Ollama democratizam o acesso à IA, permitindo que qualquer desenvolvedor execute LLMs de ponta em sua própria máquina.
- ❑ Frameworks como Langchain4j abstraem a complexidade de interagir com diferentes LLMs, oferecendo uma API unificada.
- ❑ Quando uma tecnologia se torna acessível, seu valor se desloca da **criação** para a **integração**. O desafio principal é integrar o LLM de forma segura, escalável e confiável aos processos de negócio, o que já é a especialidade do desenvolvedor Java há décadas.

APRESENTAÇÃO DA STACK TECNOLÓGICA

APRESENTAÇÃO DA STACK TECNOLÓGICA

- ❑ O curso é construído sobre três pilares tecnológicos :
 - ❑ Quarkus: O nosso runtime de aplicação. O Java “Supersônico e Subatômico”, otimizado para ambientes nativos da nuvem.
 - ❑ Langchain4j: O framework de orquestração de IA. A "cola" inteligente que conecta nossa aplicação Java aos LLMs.
 - ❑ Ollama: O servidor de LLMs local. Permite a execução de modelos offline, garantindo privacidade e agilidade.

POR QUE QUARKUS?

- ❑ Performance e Produtividade ("Developer Joy")
- ❑ Performance:
 - ❑ Baixo consumo de memória e startup em milissegundos.
 - ❑ Menor custo operacional (TCO) em nuvem e escalabilidade eficiente.
- ❑ Produtividade:
 - ❑ Live Coding: Alterações no código refletidas instantaneamente, sem reiniciar a aplicação.
 - ❑ Dev UI: Painel de controle interativo para gerenciar a aplicação em modo de desenvolvimento.
 - ❑ Dev Services: Inicia automaticamente serviços de apoio (como bancos de dados) em containers (usando TestContainers).

POR QUE LANGCHAIN4J E OLLAMA?

- ❑ Flexibilidade e Privacidade
- ❑ Langchain4j:
 - ❑ API Unificada: Abstrai a comunicação com dezenas de modelos e bancos de dados vetoriais, evitando vendor lock-in. Migrar de um modelo local para um serviço de nuvem exige apenas mudanças de configuração.
 - ❑ Ferramentas: Oferece implementações prontas para padrões como RAG e Agentes, por exemplo.
- ❑ Ollama:
 - ❑ 100% Local: Garante que nenhum dado sensível seja enviado para APIs de terceiros, um requisito crucial para muitas empresas.
 - ❑ Desenvolvimento Ágil: Elimina a latência de rede e os custos por token, permitindo experimentação rápida e ilimitada.

IMPLEMENTAÇÃO DE REFERÊNCIA: VISÃO GERAL E ARQUITETURA EVOLUTIVA

IMPLEMENTAÇÃO DE REFERÊNCIA: VISÃO GERAL E ARQUITETURA EVOLUTIVA

- ❑ O Projeto: Agente de Suporte para Agência de Viagem
 - ❑ Desenvolveremos um agente de suporte inteligente para uma agência de viagem.
 - ❑ Objetivo: Construir um chatbot que evolui de responder perguntas com base em documentos para executar ações concretas (verificar status de reserva, realizar cancelamento, etc).
- ❑ Padrões de IA abordados:
 - ❑ Q&A com RAG: Responder perguntas com base em conhecimento privado.
 - ❑ Agents & Tools: Executar ações no mundo real invocando código Java.

ARQUITETURA INICIAL

- ❑ Ponto de Partida:
 - ❑ RAG simples com Easy RAG
- ❑ Fluxo de Dados:
 1. Usuário envia pergunta (via WebSocket).
 2. Endpoint JAX-RS no Quarkus recebe a pergunta.
 3. O endpoint invoca um AiService do Langchain4j.
 4. Langchain4j usa "Easy RAG" para buscar informações em documentos de texto em memória.
 5. O prompt do usuário é "aumentado" com o contexto e enviado ao LLM no Ollama.
 6. O LLM gera uma resposta contextualizada, que é retornada ao usuário.
- ❑ Este design é 100% local, fácil de configurar e ideal para os primeiros passos.

A VISÃO EVOLUTIVA

- ❑ A arquitetura evoluirá ao longo dos módulos do curso:
 - ❑ Módulo 2 (RAG): A base de conhecimento em memória será substituída por um Banco de Dados Vetorial persistente e escalável com PgVector.
 - ❑ Módulo 3 (Agentes Inteligentes): O sistema se tornará um Agente autônomo, capaz de usar Tools (métodos Java) para interagir com outros sistemas (ex: banco de dados de reservas).
 - ❑ Módulo 4 (Integração Avançada): As Tools serão desacopladas e expostas como microservices independentes via Model Context Protocol (MCP).

PREPARAÇÃO DO AMBIENTE

PREPARAÇÃO DO AMBIENTE

❑ Checklist:

1. Instalar o Open JDK: Versão 24 ou superior.
2. Instalar o Maven
3. Instalar o Docker ou Podman
4. Instalar o Ollama: <https://ollama.com>.
5. Baixar o modelo gpt-oss:20b: `ollama pull gpt-oss:20b`

ATÉ A PRÓXIMA!